

**TEMPLATE
PROJECT WORK**

Corso di Studio	INFORMATICA PER LE AZIENDE DIGITALI (L-31)
Dimensione dell'elaborato	Minimo 6.000 – Massimo 10.000 parole (<i>pari a circa Minimo 12 – Massimo 20 pagine</i>)
Formato del file da caricare in piattaforma	PDF
Nome e Cognome	Beatrice Mollo
Numero di matricola	0312301598
Tema n. (Indicare il numero del tema scelto):	1
Titolo del tema (Indicare il titolo del tema scelto):	La digitalizzazione dell'impresa
Traccia del PW n. (Indicare il numero della traccia scelta):	16
Titolo della traccia (Indicare il titolo della traccia scelta):	Sviluppo di una applicazione full-stack API-based per un'organizzazione del settore sanitario
Titolo dell'elaborato (Attribuire un titolo al proprio elaborato progettuale):	Digitalizzazione dei processi operativi di un'OdV sanitaria: progettazione e sviluppo di un'applicazione full-stack basata su API per la gestione di richieste di trasporto, turni e mezzi

PARTE PRIMA – DESCRIZIONE DEL PROCESSO

Utilizzo delle conoscenze e abilità derivate dal percorso di studio

(Descrivere quali conoscenze e abilità apprese durante il percorso di studio sono state utilizzate per la redazione dell'elaborato, facendo eventualmente riferimento agli insegnamenti che hanno contribuito a maturarle):

Nel PW ho applicato conoscenze e le abilità acquisite nel percorso di studi in ambito informatico e digitale, con l'obiettivo di rispondere a un bisogno reale di digitalizzazione dei processi operativi all'interno di una OdV.

In particolare ho utilizzato competenze di progettazione dei sistemi informativi per trasformare un insieme di attività gestite in modo frammentato in una soluzione più strutturata, tracciabile e replicabile.

Una prima area di competenze applicate riguarda l'analisi del contesto e la raccolta dei requisiti. Ho imparato a leggere l'organizzazione non soltanto come insieme di persone e attività, ma come un sistema di processi: flussi di lavoro, decisioni, informazioni in ingresso e in uscita, criticità ricorrenti.

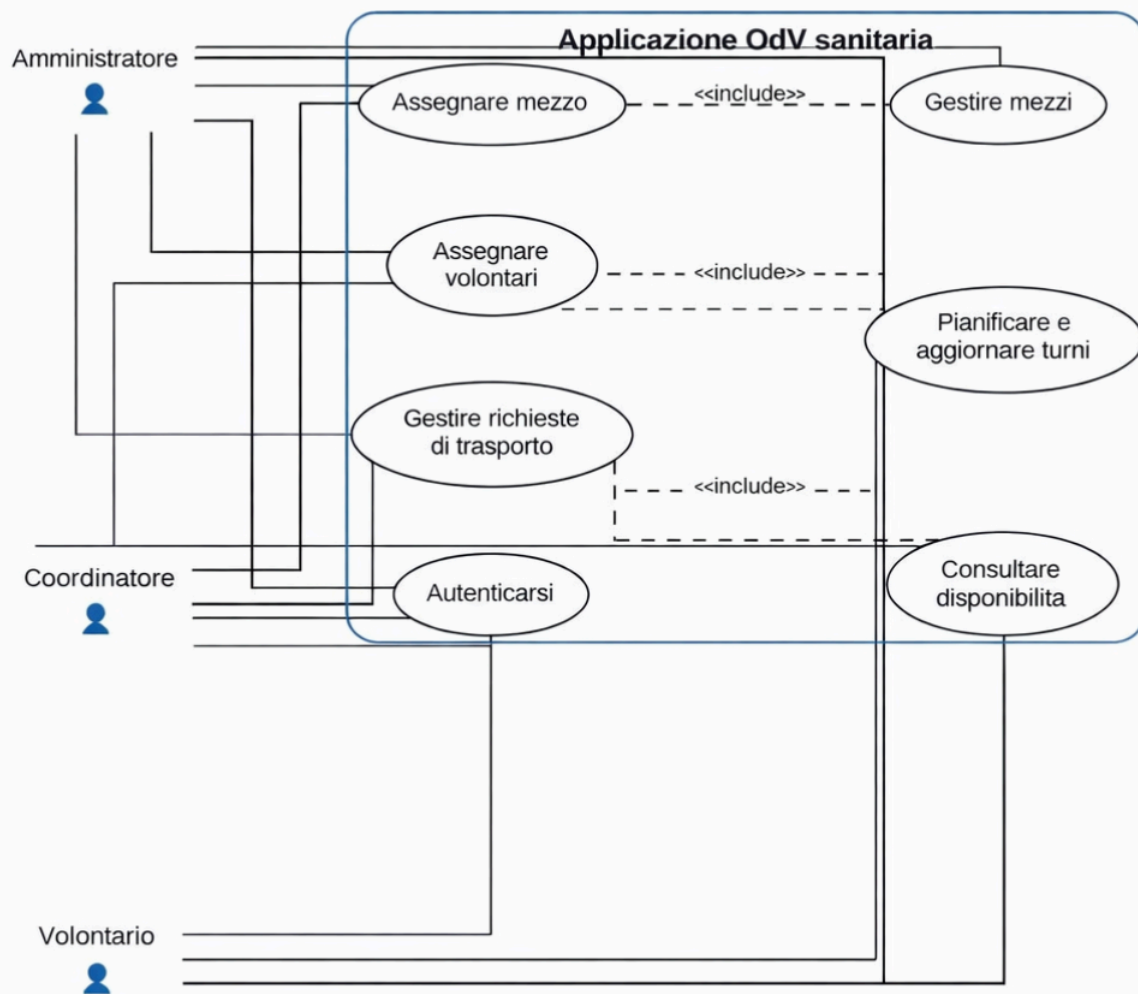
Questo approccio mi ha permesso di distinguere chiaramente tra requisiti funzionali (cosa deve fare il sistema: gestione richieste di trasporto, turni e mezzi) e requisiti non funzionali (come deve comportarsi il sistema: sicurezza, usabilità, scalabilità e attenzione al GDPR).

La definizione dei requisiti è stata fondamentale per mantenere il progetto coerente e per evitare di sviluppare funzionalità non necessarie o fuori focus.

Una seconda area riguarda la progettazione software.

Ho applicato metodi di modellazione che rendono esplicite le scelte progettuali: la rappresentazione dei casi d'uso (Use Case) per identificare attori e funzioni principali; la modellazione dei dati tramite schema Entity-Relationship (ER) per definire entità e relazioni (ad esempio turni, richieste, mezzi, utenti e relative associazioni); e la rappresentazione logica tramite Class Diagram, utile per collegare la struttura dati alla struttura applicativa.

L'utilizzo di questi strumenti consente di comunicare in modo chiaro come il sistema è pensato e quali componenti lo compongono, oltre a ridurre ambiguità durante lo sviluppo.



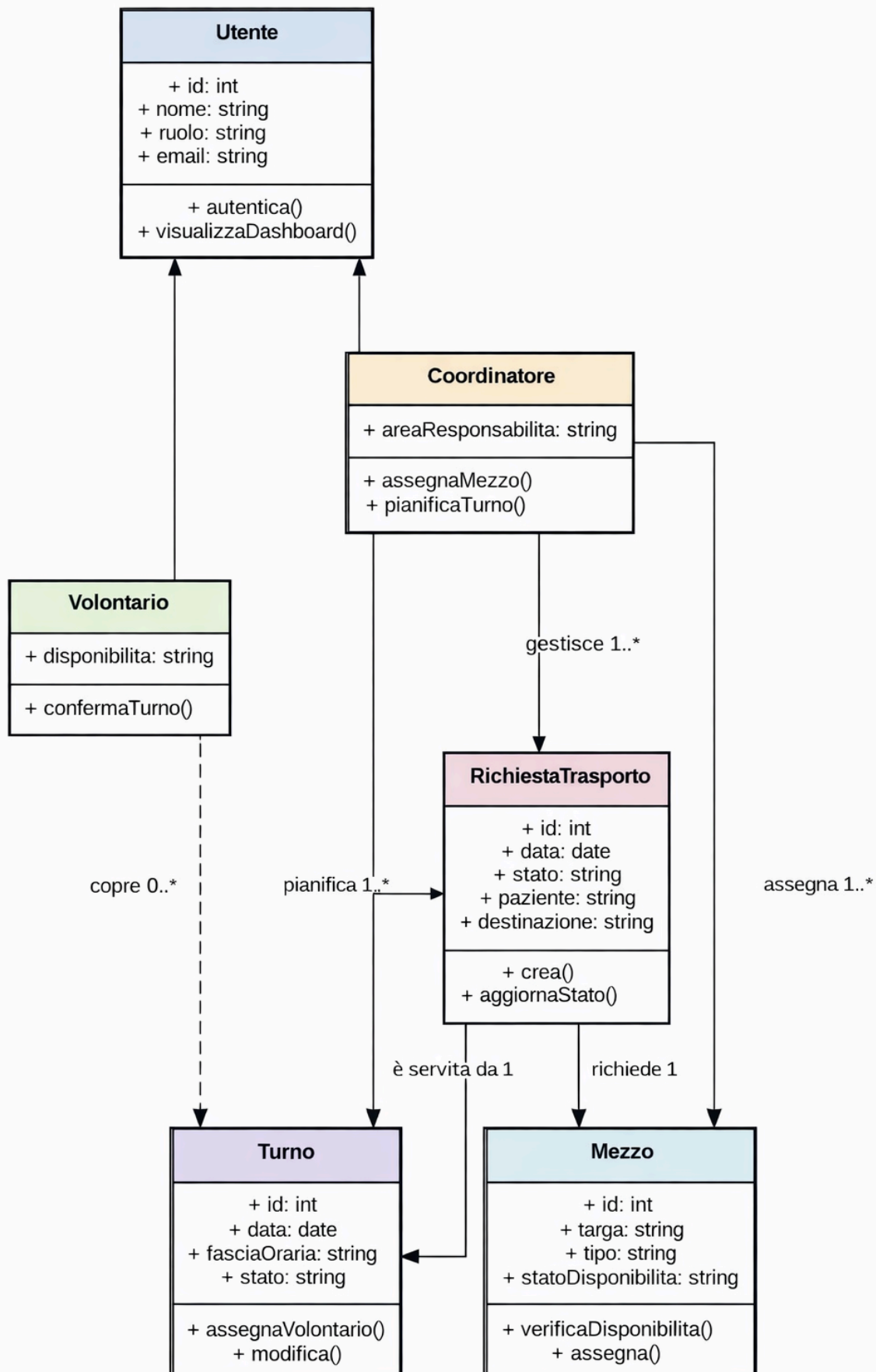
1

Figura 1 - Diagramma UML Use Case del sistema di gestione OdV sanitaria.

1

Il diagramma dei casi d'uso evidenzia gli attori principali del sistema - amministratore, coordinatore e volontario - e le funzioni centrali dell'applicazione: autenticazione, gestione delle richieste, pianificazione dei turni, consultazione della disponibilit  e assegnazione delle risorse.

¹ . In UML, il diagramma dei casi d'uso descrive il sistema dal punto di vista degli attori esterni e delle funzionalit  offerte. E' particolarmente utile nella fase iniziale di analisi dei requisiti, perch  consente di chiarire ruoli, obiettivi e interazioni principali tra utenti e applicazione.



2

² . Il Class Diagram rappresenta classi, attributi, operazioni e relazioni strutturali del dominio applicativo. Nel presente progetto aiuta a collegare la modellazione concettuale dei dati alla logica implementativa del backend e alle entità effettivamente gestita dal sistema

L'utilizzo di questi strumenti consente di comunicare in modo chiaro come il sistema è pensato e quali componenti lo compongono, oltre a ridurre ambiguità durante lo sviluppo.

Un terzo insieme di competenze riguarda la comprensione delle architetture applicative e la realizzazione di una soluzione coerente con i principi di manutenibilità.

Ho scelto un'architettura a tre livelli:

1. Frontend: si occupa di presentazione e interazione;
2. Backend: gestisce la logica e le regole, esponendo funzionalità tramite API-REST³, con endpoint coerenti con le risorse del dominio (/login, /richieste, /turni, /mezzi)
3. Database: garantisce persistenza e coerenza dei dati.

Questo approccio "API-based" rende la soluzione modulare e predisposta a evoluzioni future.

Dal punto di vista dello sviluppo, ho consolidato abilità pratiche di implementazione full-stack: da un lato la componente backend in Python (utilizzando un framework adeguato per l'esposizione di API come Flask o Fast API), dall'altro lo sviluppo del frontend in JavaScript, che dialoga con le API mediante Fetch API.

In questo modo ho potuto gestire l'intero flusso client-server: invio di richieste HTTP, gestione di payload JSON, ricezione e presentazione dei dati in interfaccia.

Un aspetto qualificante del lavoro è stata l'attenzione alla sicurezza e alla protezione dei dati. L'ambito sanitario impone cautela nella gestione delle informazioni e nella definizione degli accessi.

Per questo ho tenuto conto dei principi del GDPR⁴ (minimizzazione, controllo degli accessi, attenzione alla riservatezza e alla gestione corretta delle credenziali) come requisito non funzionale, integrandolo nella progettazione e nelle scelte implementative.

Infine, ho applicato competenze di testing e validazione.

Ho verificato le funzionalità principali del sistema sia attraverso prove operative, sia tramite strumenti come Postman, utili per testare direttamente gli endpoint e controllare che le API rispondano in modo corretto e coerente (richieste, turni, mezzi), raccogliendo evidenze di test.

In sintesi, il PW mi ha consentito di unire competenze teoriche e pratiche: analisi dei requisiti, modellazione, architettura, sviluppo full-stack, sicurezza e test, all'interno di un caso applicativo concreto.

Fasi di lavoro e relativi tempi di implementazione per la predisposizione dell'elaborato

(Descrivere le attività svolte in corrispondenza di ciascuna fase di redazione dell'elaborato. Indicare il tempo dedicato alla realizzazione di ciascuna fase, le difficoltà incontrate e come sono state superate):

Il lavoro è stato organizzato in fasi, seguendo un approccio strutturato e incrementale; dall'analisi del contesto fino alla validazione del prototipo e alla stesura finale.

Di seguito riporto le principali fasi:

1. Analisi del contesto e definizione del problema:

Ho inquadrato l'organizzazione dell'OdV e i processi operativi oggetto di digitalizzazione; gestione delle richieste di trasporto sanitario, pianificazione dei turni dei

³ API REST è uno stile architetturale per servizi web basato su risorse identificate da URL e manipolate tramite metodi HTTP. La sua adozione favorisce la separazione tra frontend e backend, la riusabilità dei servizi e la possibilità di testare gli endpoint anche in modo indipendente dall'interfaccia utente.

⁴ GDPR indica il Regolamento (UE) 2016/679 sulla protezione dei dati personali. In un contesto sanitario il suo richiamo è rilevante perché impone attenzione e minimizzazione dei dati, controllo degli accessi, riservatezza delle informazioni e corretta gestione delle credenziali

volontari e non e gestione/assegnazione dei mezzi. In questa fase ho identificato criticità tipiche della gestione manuale o frammentata (ridotta tracciabilità, difficoltà di coordinamento, rischio di duplicazioni o disallineamenti informativi).

Difficoltà: delimitare bene l'ambito per evitare che il progetto diventasse troppo ampio;

Soluzione: definire da subito i tre processi chiave (richieste/turni/mezzi) come perimetro principale;

2. Raccolta e definizione dei requisiti:

Ho tradotto i bisogni operativi in requisiti, distinguendo tra funzionali (funzione specifiche: inserimento, gestione richieste, gestione turni, gestione mezzi, autenticazione) e non funzionali (sicurezza, usabilità, saldabilità, attenzione GDPR).

Difficoltà: rendere i requisiti abbastanza chiari da guidare la progettazione e, allo stesso tempo, realistici rispetto a un prototipo.

Soluzione: organizzare i requisiti in modo gerarchico, concentrandomi su ciò che era essenziale e verificabile.

3. Progettazione e modellazione:

Ho progettato la soluzione adottando un'architettura a tre livelli e ho prodotto la documentazione progettuale:

- Diagramma Use Case (funzioni e attori);
- Schema ER (entità e relazioni);
- Class Diagram: (struttura logica applicata).

Difficoltà: mantenere coerenza tra i modelli;

Soluzione: revisione interattiva.

4. Implementazione del backend e definizione delle API:

Ho implementato il backend in Python utilizzando un framework per API (Flask o FastAPI), definendo gli endpoint principali coerenti con il progetto: /login, /richieste, /turni, /mezzi. Ho lavorato sulla logica delle operazioni principali e sulla gestione delle risposte in formato JSON.

Difficoltà: strutturare in modo ordinato endpoint e dati restituiti, evitando incongruenze.

Soluzione: definire uno schema coerente di request/response per ogni risorsa, e gestire gli errori con messaggi chiari.

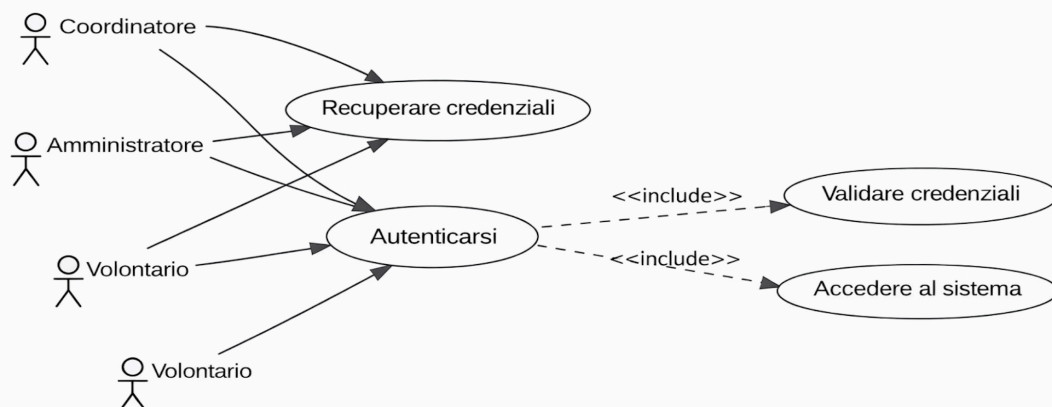


Figura 3 - Diagramma UML relativo all'autenticazione e all'accesso al sistema.

5. Implementazione del frontend e integrazione con le API:

Ho sviluppato le pagine principali in JavaScript, utilizzando la Fetch API per comunicare con il backend. Ho curato i flussi principali: login, gestione richieste/turni/mezzi.

Difficoltà: gestire correttamente la sincronizzazione tra dati lato server e visualizzazione lato client;

Soluzione: verifiche continue con test di chiamata API e controllo dei payload JSON.

6. Testing e validazione funzionale:

Ho testato le funzionalità principali con casi realisti e, soprattutto, ho utilizzato Postman per validare gli endpoint in modo indipendente dal frontend. Ho raccolto screenshot come evidenza delle chiamate e delle risposte.

Difficoltà: distinguere rapidamente problemi di backend da problemi di frontend;

Soluzione: testare prima le API con Postman, e solo dopo verificare l'integrazione via interfaccia.

7. Stesura e revisione dell'elaborato:

Ho redatto le soluzioni dell'elaborato seguendo il template richiesto; descrizione del processo, obiettivi, contestualizzazione, aspetti progettuali, campi di applicazione e valutazione dei risultati. Ho rivisto coerenza interna, terminologia e collegamenti tra requisiti, progetto e test.

Difficoltà: evitare ripetizioni e mantenere un linguaggio coerente tra parte tecnica e parte descritta;

Soluzione: strutturare il testo in paragrafi con un filo logico: contesto → requisiti → progettazione → implementazione → test → risultati.

Risorse e strumenti impiegati

(Descrivere quali risorse - bibliografia, banche dati, ecc. - e strumenti - software, modelli teorici, ecc. - sono stati individuati ed utilizzati per la redazione dell'elaborato. Descrivere, inoltre, i motivi che hanno orientato la scelta delle risorse e degli strumenti, la modalità di individuazione e reperimento delle risorse e degli strumenti, le eventuali difficoltà affrontate nell'individuazione e nell'utilizzo di risorse e strumenti ed il modo in cui sono state superate):

Per la realizzazione dell'elaborato sono state utilizzate risorse metodologiche, strumenti di progettazione, strumenti di sviluppo e strumenti di testing, selezionati in modo coerente con l'obiettivo di digitalizzare i processi operativi di una OdV sanitari tramite una soluzione full-stack API-based.

Risorse metodologiche e concettuali:

- Analisi dei processi e del contesto organizzativo: identificazione dei flussi principali (richieste di trasporto, turni, mezzi) e delle criticità legate a gestione manuale o frammentata;
- Definizione dei requisiti: distinzione tra requisiti funzionali e non funzionali, con attenzione ai vincoli di sicurezza, usabilità, scalabilità e GDPR;
- Approccio architetturale a tre livelli: separazione tra frontend, backend e database, per migliorare manutenibilità e predisposizione a evoluzioni future.

Strumenti di progettazione e modellazione:

- Diagramma Use Case per descrivere attori e funzionalità principali del sistema;

- Modello ER per progettare la base dati e rappresentare entità e relazioni;
- Class Diagram per descrivere la struttura logica del sistema e collegare dominio e implementazione.

Strumenti di sviluppo (software e tecnologie):

- Backend in Python con framework per la realizzazione di API, utilizzato per implementare la logica applicativa e gli endpoint REST;
- API REST come modalità di esposizione dei servizi, con endpoint principali /login, /richieste, /turni, /mezzi.
- Frontend in JavaScript con utilizzo della Fetch API per l'integrazione client-server, invio richieste HTTP e gestione delle risposte JSON;
- Database progettato secondo le entità individuate, coerente con modello ER e con le operazioni previste dalle API.

Strumenti di testing e validazione:

- Postman per eseguire test diretti sugli endpoint (verifica di request/response, codici di stato, correttezza dei payload), riducendo il rischio di confondere errori di backend con problemi di interfaccia;
- Raccolta di screenshot come evidenza delle prove effettuate, utile anche ai fini documentali.

Motivi della scelta e difficoltà affrontate:

Le scelte degli strumenti sono state orientate a ottenere una soluzione chiara e modulare: Python e un framework API consentono sviluppo rapido e strutturato; le API REST permettono di separare logicamente backend e frontend; JavaScript con Fetch API consente un'interfaccia leggera e integrata.

La principale difficoltà è stata mantenere allineati requisiti, progettazione e implementazione.

La difficoltà è stata superata lavorando in modo iterativo, verificando la coerenza tra diagrammi, strutture dati ed endpoint, e validando progressivamente il funzionamento tramite test.

PARTE SECONDA – PREDISPOSIZIONE DELL'ELABORATO

Obiettivi del progetto

(Descrivere gli obiettivi raggiunti dall'elaborato, indicando in che modo esso risponde a quanto richiesto dalla traccia):

L'obiettivo del progetto è la digitalizzazione dei processi operativi di una OdV del settore sanitario, individuata nel caso studio di una SOS di Varese, attraverso la progettazione e lo sviluppo di un'applicazione full-stack API.

Il sistema intende supportare in modo centralizzato e strutturato tre ambiti operativi fondamentali, strettamente interconnessi:

- Gestione delle richieste di trasporto sanitario: registrare, consultare e aggiornare le richieste in un ambiente unico, riducendo dispersione informativa, duplicazioni e rischio di perdita di dati;

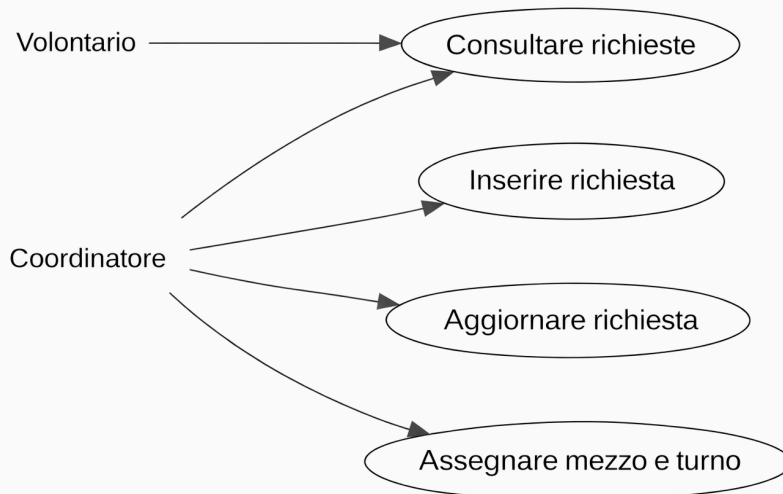


Figura 4 - Diagramma UML relativo alla gestione delle richieste di trasporto

- Gestione dei turni dei volontari: migliorare la pianificazione e la visibilità dell'organizzazione dei turni, facilitando il coordinamento interno e riducendo la necessità di comunicazioni frammentate;

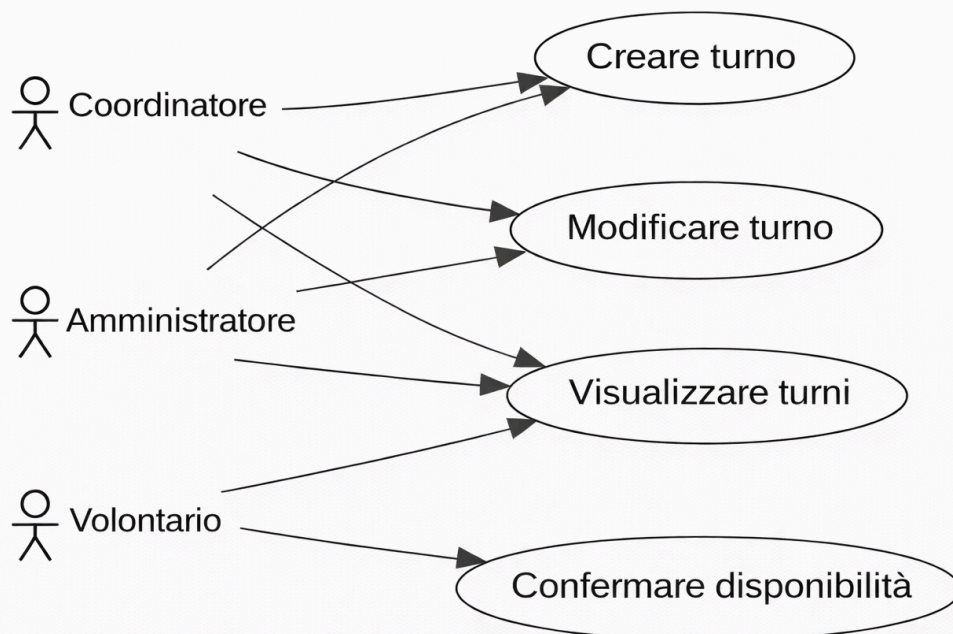


Figura 5 - Diagramma UML relativo alla gestione dei turni dei volontari.

- Gestione dei mezzi dell'associazione: mantenere un'anagrafica mezzi e supportare la disponibilità/assegnazione in modo coerente con servizi e turni, evitando sovrapposizioni e disallineamenti.

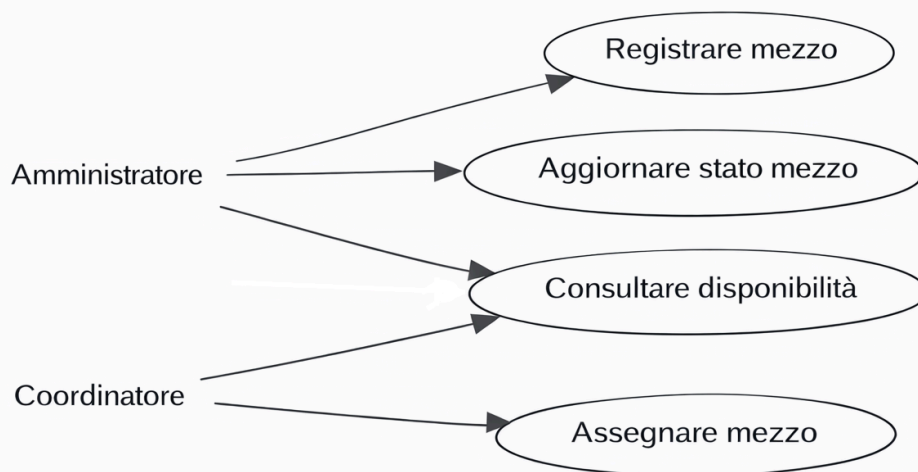


Figura 6 - Diagramma UML relativo alla gestione dei mezzi dell'associazione.

Contestualizzazione

(Descrivere il contesto teorico e quello applicativo dell'elaborato realizzato):

L'elaborato si colloca nel tema della digitalizzazione dell'impresa applicata a un'organizzazione del terzo settore: una OdV sanitaria che svolge attività operative quotidiane ad alto valore sociale. L'OdV opera in provincia di Varese (VA) e gestisce servizi di emergenza-urgenza, trasporti sanitari e attività formative e presenta esigenze tipiche di contesti in cui persone, mezzi e servizi devono essere coordinati con precisione e rapidità.

Dal punto di vista teorico, la digitalizzazione dei processi può essere intesa come il passaggio da una gestione informale, manuale o frammentata a una gestione strutturata attraverso un sistema informativo, con benefici come maggiore tracciabilità, standardizzazione delle procedure, riduzione degli errori, migliore accessibilità ai dati e capacità di controllo.

Questo tema è particolarmente rilevante per organizzazioni in cui le informazioni operative possono essere distribuite tra strumenti diversi (messaggistica, file non integrati, annotazioni), con il rischio di perdita o incoerenza delle informazioni.

Dal punto di vista applicativo, l'elaborato traduce questi principi in una soluzione concreta: una web application full-stack basata su API, progettata per centralizzare tre processi cruciali dell'operatività: richieste di trasporto, turni e mezzi.

La scelta dell'approccio API-based e dell'architettura a tre livelli risponde a critiche di modularità e manutenibilità, separando l'interfaccia utente dalla logica applicativa e dalla persistenza, la soluzione può essere estesa nel tempo senza dover essere progettata integralmente.

Descrizione dei principali aspetti progettuali
(Sviluppare l'elaborato richiesto dalla traccia prescelta):

Gli aspetti progettuali del sistema sono stati sviluppati a partire dai requisiti e organizzati secondo una logica di coerenza tra dominio applicativo, architettura tecnica, modello dati e servizi esposti.

Architettura della soluzione:

Il sistema è progettato secondo un'architettura a tre livelli:

1. Frontend: componente di presentazione che consente agli utenti di interagire con il sistema tramite interfaccia web;
2. Backend: componente applicativa che implementa la logica e mette a disposizione funzionalità tramite API REST;
3. Database: componente di persistenza in cui vengono memorizzate informazioni relative a richieste, turni, mezzi e utenti.

Questa architettura consente di separare responsabilità e semplificare manutenzione ed evoluzione, il frontend può cambiare senza modificare la logica, e il backend può essere esteso con nuove funzioni senza ricostruire l'interfaccia da zero.

Modellazione e documentazione progettuale:

Per rendere esplicita e verificabile la progettazione sono stati utilizzati strumenti di modellazione:

- Use Case: identificazione degli attori come amministratori, coordinatori e/o volontari e delle funzionalità principali come gestione richieste, turni, mezzi, autenticazione.
- Schema ER: definizione delle entità e delle relazioni per la base dati, coerenti con le risorse gestite dal sistema;
- Class Diagram: rappresentazione della struttura logica applicativa per collegare i dati alle classi/oggetti software.

Progettazione delle API e degli endpoint:

La soluzione espone servizi tramite API REST. Gli endpoint principali del progetto sono:

- login per l'autenticazione e l'accesso al sistema;
- richieste per la gestione delle richieste di trasporto, inserimento, consultazione e aggiornamento;
- turni per la gestione dei turni dei volontari;
- mezzi per la gestione dell'anagrafica e delle informazioni operative sui mezzi.

Le API restituiscono dati in formato JSON e consentono al frontend di operare tramite chiamate HTTP.

Questo approccio permette test indipendenti del backend e favorisce l'integrazione futura con altri client.

Scelte implementative:

Il backend è sviluppato in Python utilizzando un framework adeguato per API.

Il frontend è realizzato in JavaScript e comunica con il backend tramite Fetch API.

Sicurezza, GDPR e requisiti non funzionali:

Il progetto integra requisiti non funzionali rilevanti per il contesto sanitario, in particolare:

- Controllo degli accessi e autenticazione;
- Attenzione alla protezione dei dati e ai principi GDPR;

- Usabilità;
- Scalabilità e predisposizione a evoluzioni future.

Validazione e testing:

Gli aspetti progettuali sono stati verificati attraverso test delle funzionalità principali e attraverso l'uso del Postman per il testing degli endpoint, con raccolta di evidenze.

Questa fase ha confermato la coerenza tra requisiti, progettazione e comportamento delle API.

Campi di applicazione

(Descrivere gli ambiti di applicazione dell'elaborato progettuale e i vantaggi derivanti della sua applicazione):

L'applicazione sviluppata ha un campo di applicazione immediato nel settore del volontariato sanitario e, più in generale, in tutte le organizzazioni che gestiscono servizi di trasporto, turnazione del personale/volontari e risorse materiali.

Il modello proposto è particolarmente adatto a contesti in cui è necessario coordinare più elementi contemporaneamente, una richiesta operativa richiede la disponibilità di persone e mezzi, e la gestione efficace dipende dalla possibilità di accedere rapidamente a informazioni aggiornate e coerenti.

Ambiti di applicazione principali:

- OdV e associazioni di volontariato sanitario: gestione di trasporti sanitari programmati e attività di supporto, con pianificazione dei turni e assegnazione dei mezzi;
- Associazioni socio-assistenziali: organizzazione di servizi di accompagnamento e supporto con necessità di coordinare richieste e risorse;
- Organizzazioni con team distribuiti e disponibilità variabile: realtà in cui le persone non hanno presenza fissa e la turnazione è un elemento centrale;
- Contesti operativi che richiedono tracciabilità: quando diventa importante ricostruire attività, assegnazioni e aggiornamenti in modo ordinato.

Vantaggi derivanti dall'applicazione:

- Centralizzazione delle informazioni: richieste, turni e mezzi sono gestiti in un unico sistema riducendo dispersione e duplicazione;
- Migliore coordinamento: la visibilità sulle richieste e sulle risorse disponibili facilita l'organizzazione operativa e riduce tempi di risposta;
- Riduzione degli errori: dati strutturati e processi digitali riducono incoerenze dovute a trascrizioni o comunicazioni non standardizzate;
- Tracciabilità: la gestione tramite applicazione consente una migliore ricostruzione delle attività e degli argomenti rispetto a strumenti informali;
- Scalabilità: grazie all'approccio API-based, la soluzione può essere estesa;
- Miglioramento della gestione dei dati: l'attenzione ai requisiti di sicurezza e GDPR rende più controllato l'accesso alle informazioni e favorisce una gestione più responsabile.

Il modello rappresenta una base replicabile, partendo dal caso SOS, può essere adattato ad altre realtà simili.

Valutazione dei risultati

(Descrivere le potenzialità e i limiti ai quali i risultati dell'elaborato sono potenzialmente esposti)

La valutazione dei risultati dell'elaborato evidenzia come l'approccio proposto produca benefici concreti soprattutto in termini di efficienza organizzativa, coordinamento e qualità della gestione delle informazioni.

Potenzialità e punti di forza:

1. Miglioramento dell'efficienza operativa: centralizzare richieste, turni e mezzi riduce il tempo necessario per reperire informazioni e prendere decisioni operative.
Il coordinamento o l'operatore può consultare rapidamente lo stato delle richieste e le risorse disponibili, riducendo passaggi manuali e comunicazioni frammentate;
2. Maggiore coordinamento tra processi collegati: il valore principale deriva dall'integrazione di tre aree che sono interdipendenti.
Gestire in un unico sistema le ricerche di trasporto, la turnazione e i mezzi rende più semplice collegare esigenze e risorse, con minori rischi di disallineamento.
3. Tracciabilità e riduzione del rischio di errore: un sistema digitale introduce dati strutturati e standardizzati e permette di ridurre problemi tipici delle gestioni manuali;
4. Modularità e predisposizione all'evoluzione: l'architettura a tre livelli e la scelta API-based rendono la soluzione più manutenibile e scalabile.
In futuro sarà possibile estendere le funzionalità senza riscrivere l'intero sistema;
5. Attenzione a sicurezza e GDPR: integrare requisiti non funzionali rappresenta un miglioramento importante rispetto a soluzioni non progettate per gestire informazioni operative in un contesto sanitario.

Verifica e coerenza con i requisiti:

Le funzionalità principali sono state validate attraverso test mirati su gestione richieste, turni e mezzi.

L'utilizzo di Postman per il testing degli endpoint ha permesso di verificare le API in modo indipendente dal frontend e di confermare la correttezza delle operazioni fondamentali, con evidenze raccolte tramite screenshot.

Limiti e aspetti migliorabili:

Come ogni prototipo o prima versione, il sistema presenta limiti che rappresentano aree di evoluzione:

- Hardening per uso in produzione: per un utilizzo reale esteso sarebbero necessari ulteriori controlli di sicurezza, gestione più avanzata delle autorizzazioni e procedure consolidate (backup, gestione utenti più dettagliata);
- Interfaccia e user experience: l'usabilità è un requisito centrale, ulteriori interazioni potrebbero rendere l'interfaccia più completa e ottimizzata per diversi dispositivi;
- Funzionalità aggiuntive: integrazioni con notifiche, reportistica e strumenti di analisi dati possono aumentare ulteriormente il valore, in linea con gli sviluppi futuri già individuati;
- Logging e audit: in ambito operativo può essere utile una tracciatura più strutturata delle azioni per finalità organizzative e di controllo.

Conclusione valutativa:

Nel complesso, i risultati indicano che la situazione è coerente con la traccia e con il caso applicativo, il sistema digitalizza in modo strutturato processi operativi essenziali di una OdV sanitaria, mostrando come un approccio full-stack API-based possa migliorare efficienza, coordinamento e affidabilità nella gestione quotidiana.

Riferimento al repository del codice

Il codice sviluppato per il presente progetto è disponibile al seguente repository⁵ Github:
<https://github.com/beatricemollo15-mb/Sviluppo-di-una-applicazione-full-stack-API-based-per-un-organizzazione-del-settore-sanitario.git>

Bibliografica

- Fowler, M. (2004). UML Distilled: guida sintetica al linguaggio standard di modellazione a oggetti (3^a ed.). Addison-Wesley.
- Booch, G., Rumbaugh, J. e Jacobson, I. (2005). Guida utente al linguaggio UML (2^a ed.). Addison-Wesley.
- Sommerville, I. (2016). Ingegneria del software (10^a ed.). Pearson.
- Elmasri, R. e Navathe, S. B. (2017). Sistemi di basi di dati: fondamenti e progettazione (7^a ed.). Pearson.
- Richardson, L. e Ruby, S. (2007). Servizi Web RESTful. O'Reilly Media.
- Kurose, J. F. e Ross, K. W. (2021). Reti di calcolatori e Internet: un approccio top-down (8^a ed.). Pearson.
- Parlamento europeo e Consiglio dell'Unione europea. (2016). Regolamento (UE) 2016/679 del 27 aprile 2016 relativo alla protezione delle persone fisiche con riguardo al trattamento dei dati personali (GDPR).

Sitografia

- Università Telematica Pegaso, Corso di Laurea L-31 "Informatica per le Aziende Digitali" - pagina del corso e piano di studi.
- Università Telematica Pegaso, Regolamento del Corso di Laurea L-31 "Informatica per le Aziende Digitali".
- Object Management Group (OMG), Specifica del linguaggio UML, versione 2.5.1.
- Documentazione ufficiale di FastAPI.
- Centro di apprendimento Postman.
- MDN Web Docs - API Fetch.
- Documentazione Python - Python 3.
- EUR-Lex - testo ufficiale del Regolamento (UE) 2016/679 (GDPR).
- Repository del progetto:
<https://github.com/beatricemollo15-mb/Sviluppo-di-una-applicazione-full-stack-API-based-per-un-organizzazione-del-settore-sanitario.git>

Ringraziamenti

⁵ Il repository raccoglie il prototipo sviluppato e il codice di supporto descritto nell'elaborato. Costituisce un riferimento utile sia per documentare le scelte implementative sia per mostrare in modo concreto la struttura del progetto, degli endpoint e l'organizzazione delle componenti software